Reviewer Pack — Minimal Order of Artinian Rings and Resolution Proof Width I…

Entry 2a200936cac6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 20:24:09 UTC

Minimal Order of Artinian Rings and Resolution Proof Width Inequality

Entry ID: `2a200936cac6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 20:24:09 UTC

1. Conjecture

Field A (mathematical branch): Ring Theory (Artinian Rings) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every Boolean satisfiability instance φ, the minimal number of generators of its associated Artinian ring A(φ) is linearly correlated with its resolution proof width w(φ), such that #Gen(A(φ)) = Ω(w(φ)). Additionally, for any instance φ with a resolution proof width greater than a certain threshold k, there exists an associated Artinian ring A(φ) with #Gen(A(φ)) ≥ 2^k.

Rationale (proposer's reasoning):

> Artinian rings provide a natural algebraic structure to study the complexity of Boolean satisfiability problems. The minimal number of generators for an Artinian ring reflects its simplicity and can potentially be used as a complexity measure. This conjecture aims to explore the relationship between the algebraic structure of Artinian rings and the resolution proof width, which is a central complexity measure in propositional logic.

Taxonomy category: `Ring_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `68cf3b5872c43151`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 24 out of 30 seeds yield #Gen(A(φ)) ≥ 2^k for all instances φ with w(φ) > k and a correlation coefficient r ≥ 0.8 between #Gen(A(φ)) and w(φ). The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Artinian rings" AND "resolution proof complexity" - "minimal number of generators" AND artinian ring resolution width" - "#Gen(A(φ)) >= 2^k" AND associated Artinian ring

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def rank(A):
        m, n = len(A), len(A[0])
        row echelon_form = gaussian_elimination(A)
        rank = 0
        for i in range(m):
            if any(echelon_form[i][j] != 0 for j in range(n)):
                rank += 1
        return rank
```

```
    def resolution_width(phi):
        # Placeholder function to simulate resolution width calculation
        # Replace with actual implementation
        return random.randint(5, 20)

    def artinian_ring_generators(phi):
        # Placeholder function to simulate Artinian ring generators calculation
        # Replace with actual implementation
        n = len(phi)
        A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        return rank(A)

    instances_tested = 0
    n_max = 0
    total_gen = 0
    total_width = 0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            phi = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
            gen = artinian_ring_generators(phi)
            width = resolution_width(phi)
            instances_tested += 1
            n_max = max(n_max, n)
            total_gen += gen
            total_width += width

    mean_gen = total_gen / instances_tested
    mean_width = total_width / instances_tested
    correlation_coefficient = (instances_tested * total_gen * total_width -
                               sum(gen * width for gen, width in zip(generators, widths))) / \
                              math.sqrt((instances_tested * sum(gen**2 for gen in generators) - su
                                         (instances_tested * sum(width**2 for width in widths) -

    conjecture_holds = correlation_coefficient >= 0.8 and all(g >= 2**w for g, w in zip(generators
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
mean_gen = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_gen} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={next(se
else:
    print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11895
2	propose	ollama_remote	glm4:latest	0	0	13961
3	propose	ollama_remote	glm4:latest	0	0	16575
4	propose	ollama_remote	glm4:latest	0	0	13656
5	preregistration	ollama_remote	glm4:latest	0	0	10213
6	novelty	ollama_remote	glm4:latest	0	0	10837
7	novelty	ollama_remote	glm4:latest	0	0	9323
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15856
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8010
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9970
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12445
12	judge	ollama_remote	glm4:latest	0	0	11642

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 144382 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/2a200936cac6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2a200936cac6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2a200936cac6.tar.gz` (if generated)